

Rozwój adaptacyjnego agenta RL do dostrajania regulatora PID silnika BLDC

Dawid Robak, Kacper Sułkowski

Czerwiec 2026

1 Wstęp i modelowanie matematyczne obiektu regulacji (10.03.2026 – 24.03.2026)

Głównym celem niniejszego projektu było zaprojektowanie oraz implementacja agenta sztucznej inteligencji zdolnego do adaptacyjnego dostrajania regulatora PID sterującego bezszczotkowym silnikiem prądu stałego (BLDC). Kluczowym wymaganiem stawianym przed opracowywanym systemem była zdolność do ciągłej obserwacji i dynamicznego reagowania na nieliniowe, zmienne warunki środowiskowe, takie jak wahania obciążenia zewnętrznego, zmiany indukcyjności czy sił tarcia.

Projekt zrealizowano w środowisku uruchomieniowym Python 3, wykorzystując biblioteki:

- **control** – do analizy układów automatyki i wyznaczania odpowiedzi,
- **gymnasium** – do konstrukcji znormalizowanego środowiska uczącego,
- **optuna** – do przeprowadzenia procesów optymalizacji hiperparametrów oraz wag funkcji celu.

Jako bazowe podejście aplikacyjne wybrano algorytmy uczenia ze wzmocnieniem (*Reinforcement Learning*), w szczególności algorytm Proximal Policy Optimization (PPO), wspierany w fazie tuningu przez metody optymalizacji bayesowskiej.

Prace rozpoczęto od zdefiniowania podstawowej architektury systemu oraz stworzenia matematycznego opisu silnika BLDC. W początkowej fazie zaimplementowano model ciągły, w którym uwzględniono zaszumienie sygnałów prądowych, prędkości obrotowej oraz napięcia zasilania, co miało na celu zbliżenie symulacji do warunków rzeczywistych. Równolegle opracowano strukturę klasycznego regulatora PID i zintegrowano ją ze stworzonym środowiskiem Gymnasium.

Pierwsze testy walidacyjne wykazały jednak konieczność przebudowy jądra obliczeniowego. W celu uproszczenia kodu i skrócenia czasu obliczeń, reprezentację opartą na transformatach operatorowych (transmitancjach) zastąpiono jawnym opisem w przestrzeni stanów (*State-Space*). Wykonywanie odczytów poboru prądu bezpośrednio z macierzy stanu znacząco podniosło stabilność numeryczną. Dodatkowo, w strukturze regulatora PID zaimplementowano algorytm *anti-windup* (obcinanie składnika całkującego) oraz wprowadzono parametryzację limitów sygnału wyjściowego.

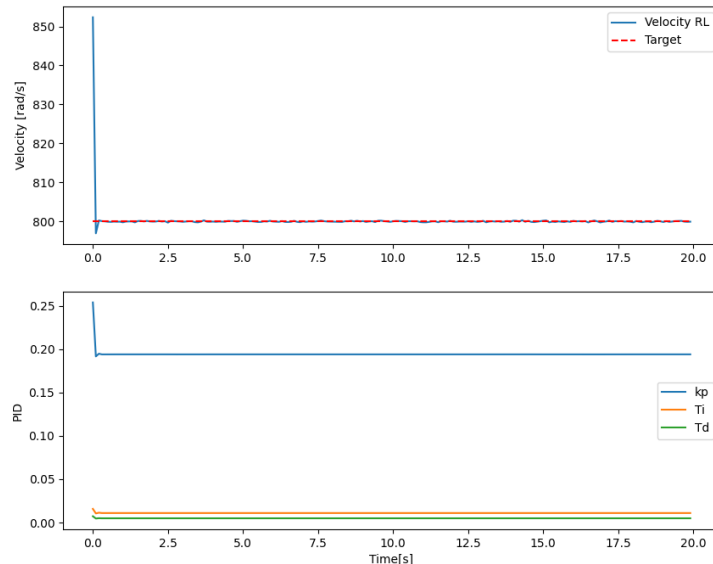
2 Dostrajanie środowiska symulacyjnego i stabilizacja regulacji (25.03.2026 – 31.03.2026)

Istotnym krokiem w rozwoju symulatora było zbliżenie jego parametrów do fizycznego obiektu. Jako model referencyjny przyjęto silnik bezszczotkowy stosowany w dronach (model 2212). Do układu wprowadzono zewnętrzny moment obciążający jako niezależny sygnał wejściowy. Równocześnie dokonano modyfikacji samej struktury regulatora, przechodząc z konfiguracji niezależnej (IND) na architekturę zgodną ze standardem ISA (*International Society of Automation*).

Podczas wczesnych testów zauważono, że pierwotna rozdzielczość czasowa symulacji z krokiem $dt = 0,01$ s była zbyt niska, co prowadziło do niestabilności działania regulatora PID i generowania gwałtownych oscylacji. Problem ten rozwiązano poprzez skrócenie kroku dyskretyzacji do poziomu $dt = 0,001$ s. Zmiana ta, w połączeniu z przejściem z ciągłej odpowiedzi wymuszonej na pełną dyskretyzację dziedziny czasu, zaowocowała około dziesięciokrotnym przyspieszeniem obliczeń symulacyjnych.

W celu poprawy zbieżności algorytmów uczenia, zakresy obserwacji oraz akcji agenta zostały przeskalowane do znormalizowanego przedziału od -10 do 10 . Wprowadzono także skalowanie funkcji nagrody,

aby uniknąć problemu eksplodujących gradientów. Początkowe próby strojenia przy użyciu stałych wag nagród skutkowały ustaleniem się oscylacji wokół wartości zadanej na poziomie $\pm 10\%$. Dopiero precyzyjne przeważenie kary za błąd regulacji oraz nagrody za wysoką dokładność pozwoliło na uzyskanie stabilnej pracy układu w klasycznej konfiguracji nieadaptacyjnej (Rysunek 1).



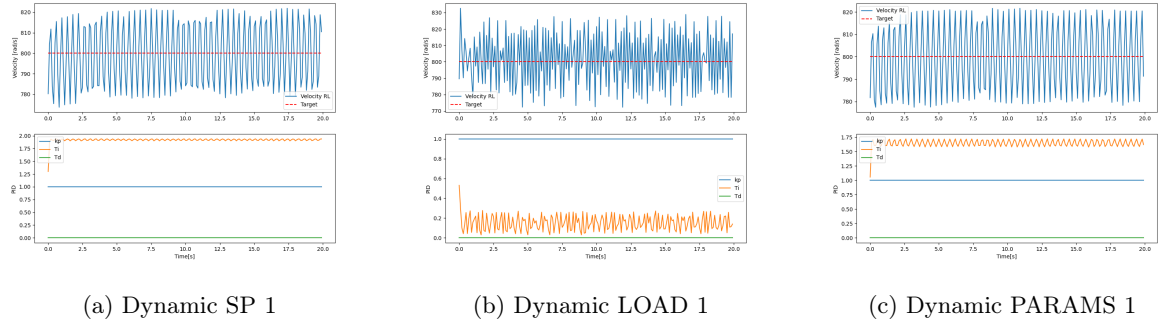
Rysunek 1: Odpowiedź układu dla dynamicznej wartości zadanej (Dynamic SP 1) przy bazowych nastawach nienadających.

3 Implementacja środowiska dynamicznego i trening agenta RL (01.04.2026 – 05.05.2026)

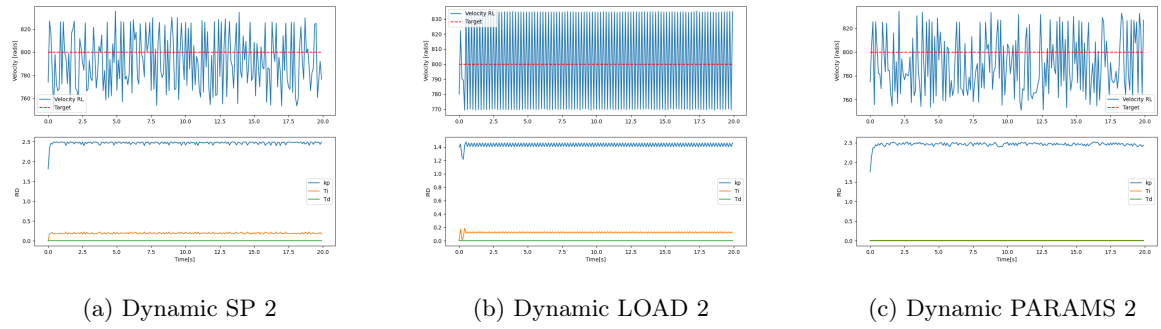
Po stabilizacji modelu matematycznego przystąpiono do realizacji zaawansowanych scenariuszy treningowych. Wprowadzono mechanizmy losowej i dynamicznej zmiany trzech kluczowych komponentów procesu: wartości zadanej prędkości obrotowej (*Dynamic SP*), momentu obciążenia (*Dynamic LOAD*) oraz wewnętrznych parametrów konstrukcyjnych silnika (*Dynamic PARAMS*). Uruchomienie wieloprocusowości (wykorzystanie wielu rdzeni CPU) pozwoliło na jednoczesne prowadzenie wielu sesji treningowych na jednym współdzielonym modelu, co drastycznie skróciło czas optymalizacji sieci neuronowej.

Pierwsze próby (Rysunek 2) wykazały, że bezpośrednie dyktowanie nastaw regulatora K_p , T_i , T_d przez agenta wywołuje skrajną niestabilność układu. Przełomem okazała się zmiana filozofii interakcji: akcję agenta zdefiniowano nie jako wartość bezwzględna, lecz jako procentową zmianę aktualnych nastaw. Zabieg ten natychmiast ustabilizował proces sterowania. Dodatkowo rozszerzono dopuszczalne granice poszukiwań parametrów PID do wartości maksymalnych wynoszących odpowiednio: $K_p = 10,0$, $T_i = 0,5$ oraz $T_d = 0,01$ (Rysunek 3).

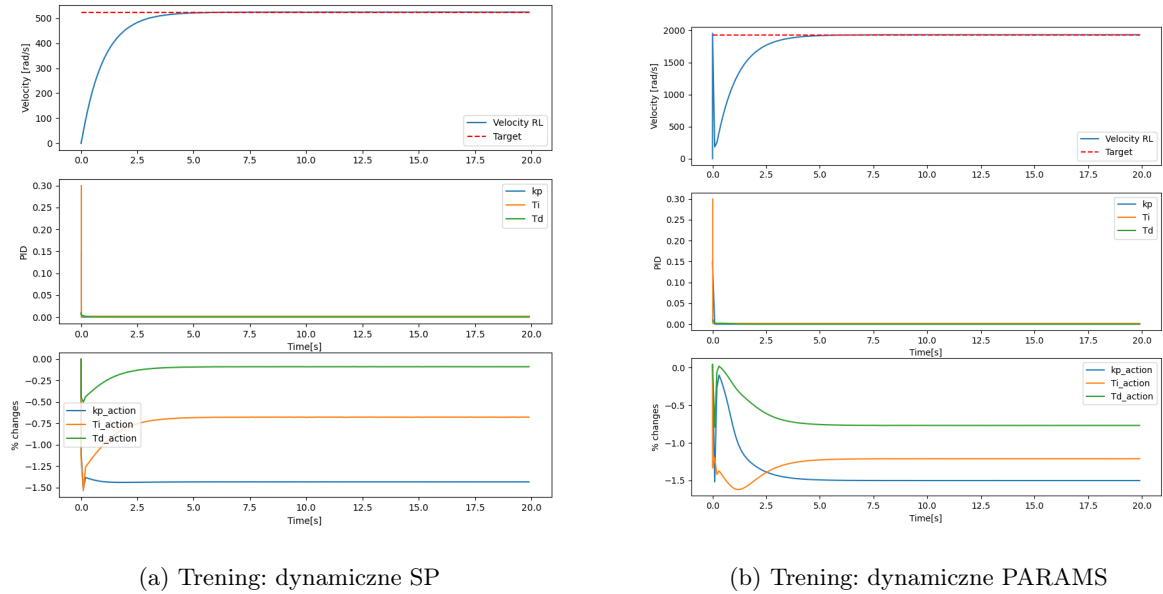
Weryfikacja modelu po wprowadzeniu procentowej formy zadawania akcji oraz losowości sygnałów wejściowych (Rysunek 4) potwierdziła znaczący wzrost szybkości nadążania za zmianami. W celu zbudowania pełnej odporności agenta na nieprzewidziane anomalie, opracowano sekwencyjny plan treningowy w ramach jednej epoki (trwającej 30 sekund od resetu do resetu). Przez pierwszą jedną trzecią czasu iteracji agent uczył się reagować na skokowe zmiany wartości zadanej (SP), w drugiej fazie dołączano zmienność parametrów silnika, a w ostatniej fazie wprowadzano dynamiczne fluktuacje obciążenia. Zaimplementowano również losowość znaczników czasu, w których dochodziło do zmian wymuszeń. Przeprowadzone modyfikacje pozwoliły na uzyskanie znacznie ściślejszej i szybszej regulacji w środowisku niedeterministycznym (Rysunek 5).



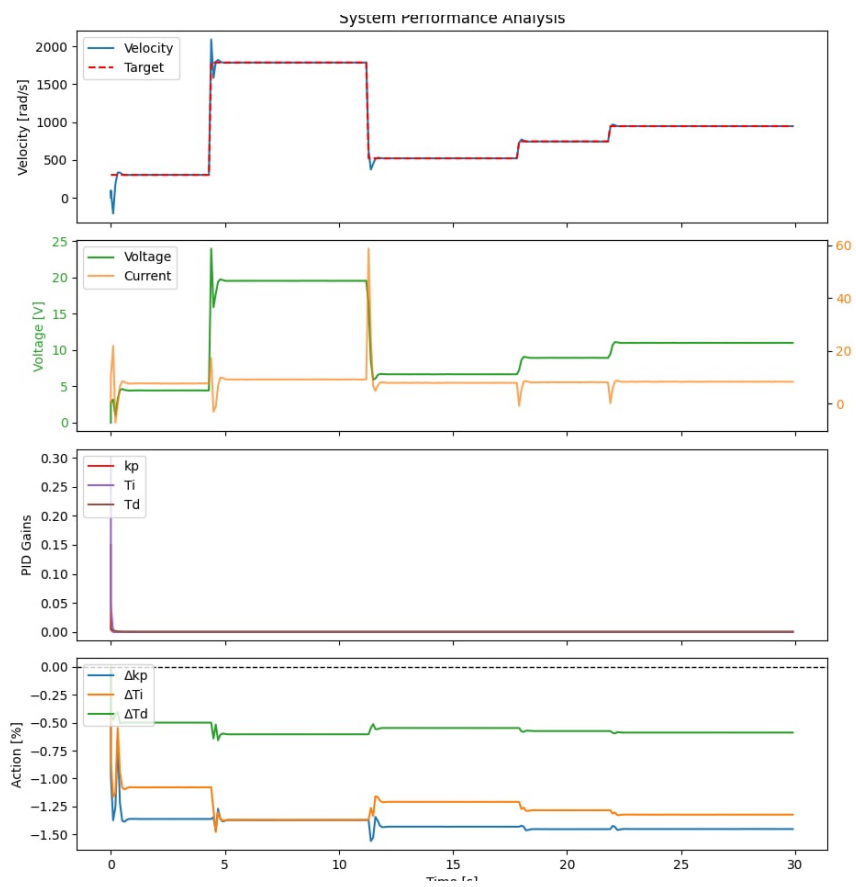
Rysunek 2: Wstępne sesje treningowe (Wersja 1) wykazujące niestabilne zachowanie agenta RL.



Rysunek 3: Eksperymenty treningowe (Wersja 2) po rozszerzeniu limitów granicznych parametrów PID.



Rysunek 4: Odpowiedzi układu testowego przy losowych wymuszeniach wartości zadanej oraz parametrów konstrukcyjnych.

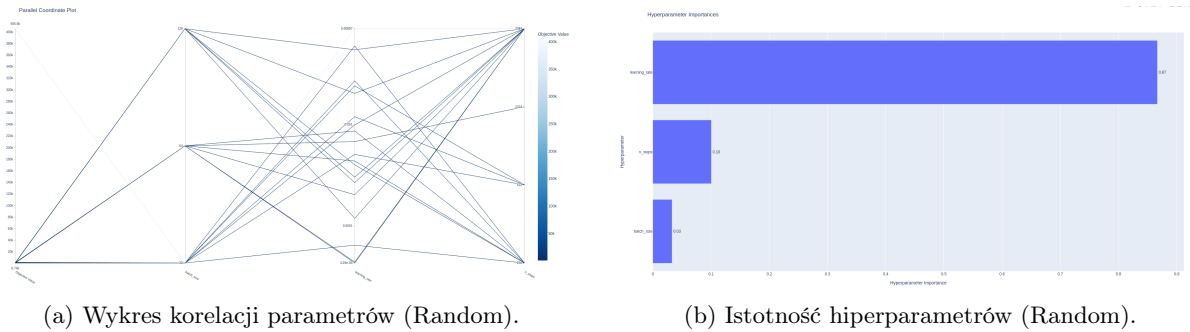


Rysunek 5: Progres jakości regulacji prędkości silnika BLDC w środowisku niedeterministycznym pod koniec etapu.

4 Optymalizacja hiperparametrów algorytmu PPO (06.05.2026 – 12.05.2026)

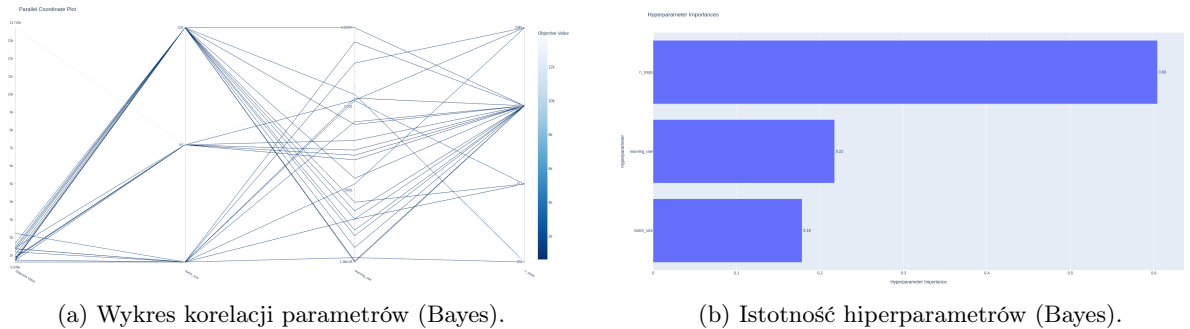
W celu maksymalizacji potencjału algorytmu PPO, zaimplementowano procedurę automatycznego wyszukiwania optymalnych hiperparametrów uczenia (szybkości uczenia – `learning_rate`, długości horyzontu – `n_steps` oraz rozmiaru paczki danych – `batch_size`). Jako kryterium oceny (funkcję kosztu) przyjęto wskaźnik ITAE wyznaczany podczas ujednoliconej procedury ewaluacyjnej. Przeprowadzono porównanie dwóch strategii przeszukiwania przestrzeni parametrów: losowej (*Random Search*) oraz bayesowskiej (*TPESampler*).

Strategia *Random Search* wskazała jako najlepszy zestaw: `learning_rate` = $6,445 \times 10^{-5}$, `n_steps` = 256 oraz `batch_size` = 32. Analiza istotności parametrów wykazała wówczas, że kluczowa dla sukcesu była szybkość uczenia (70% wpływu), przed długością horyzontu (28%) (Wizualizacja korelacji i istotności na Rysunku 6).



Rysunek 6: Wyniki optymalizacji przestrzeni hiperparametrów PPO przy użyciu metody Random Search.

Znacznie lepsze rezultaty przyniósł algorytm optymalizacji bayesowskiej (*Bayes OPT*), osiągając optymalny (niższy) wskaźnik kosztu na poziomie **635,6678**. Bayesowski TPE wskazał następujące nastawy: `learning_rate` = 0,00127, `n_steps` = 1024 oraz `batch_size` = 32. W tym przypadku analiza istotności przypisała aż 80% wagi parametrowi `n_steps` (Rysunek 7).



Rysunek 7: Wyniki optymalizacji hiperparametrów algorytmu PPO przy użyciu metody Bayes OPT.

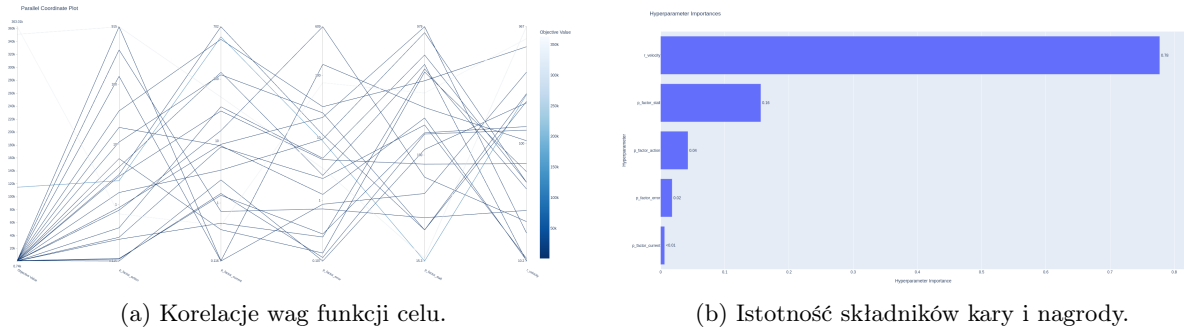
4.1 Interpretacja wyników optymalizacji hiperparametrów

Wyniki te niosą istotną wartość interpretacyjną dla teorii sterowania i uczenia maszynowego. Wymóg wysokiej wartości `n_steps` (1024) dowodzi, że do precyzyjnego kontrolowania dynamiki silnika agent potrzebuje dłuższego horyzontu czasowego, by móc właściwie zidentyfikować trendy i powiązania fizyczne układu. Z kolei mała optymalna wielkość paczki (`batch_size` = 32) potwierdza, że model uczy się efektywniej, gdy analizuje mniejsze, bardziej zróżnicowane porcje danych, co zapobiega przedwczesnemu utknięciu w minimach lokalnych.

Podejście bayesowskie pozwoliło na zastosowanie wyższego współczynnika `learning_rate` w stosunku do *Random Search*, co przełożyło się na dynamiczniejszą i skuteczniejszą adaptację sieci, podczas gdy *Random Search* wymuszał powolny, bardzo zachowawczy, a przez to mniej efektywny proces zbieżności.

5 Optymalizacja wag funkcji nagrody (13.05.2026 – 19.05.2026)

Kolejnym etapem optymalizacji było dostrojenie wag wewnątrz samej funkcji nagrody środowiska, opisanego obiektem `AimFuncParams`. Parametry te bezpośrednio definiują zachowanie agenta, karząc go za błędy i nagradzając za pożądane stany dynamiczne. Wykorzystując ponownie algorytm *Bayes OPT*, poddano tuningowi pięć współczynników w skali logarytmicznej: karę za błąd prędkości (`p_factor_error`), karę za nadmierny prąd (`p_factor_current`), karę za gwałtowne ruchy sterujące regulatora (`p_factor_action`), karę za utyk silnika (`p_factor_stall`) oraz nagrodę za utrzymywanie prędkości docelowej (`r_velocity`). Wyniki optymalizacji przedstawiono na Rysunku 8.



Rysunek 8: Analiza bayesowskiej optymalizacji składników funkcji celu (Aim Function Optimization).

W wyniku eksperymentu uzyskano najlepszy wskaźnik ITAE równy **743,245** dla następujących wartości parametrów:

- `p_factor_error` = 25,3113 (wartość nominalna: 10),
- `p_factor_current` = 117,5356 (wartość nominalna: 1),
- `p_factor_action` = 10,9939 (wartość nominalna: 1),
- `p_factor_stall` = 26,5091 (wartość nominalna: 200),
- `r_velocity` = 226,1838 (wartość nominalna: 100).

5.1 Interpretacja fizyczna dostrojonej funkcji celu

Zestawienie to dostarcza cennych wniosków inżynierskich. Zwiększenie kary za błąd prędkości o 2,5 raza zmusza agenta do dążenia do maksymalnej dokładności w stanach ustalonych. Ekstremalny wzrost kary za prąd (z 1 do ponad 117) idealnie odzwierciedla fizykę rozruchu silnika BLDC. Ponieważ prąd gwałtownie rośnie w stanach przejściowych, wysoka restrykcja w tym obszarze wymusza na agencie takie kształtowanie nastaw PID, aby czas trwania tych stanów był jak najkrótszy, co bezpośrednio minimalizuje skumulowany w czasie błąd ITAE.

Wzrost kary za akcję sterującą (`p_factor_action`) skutecznie przeciwdziała zjawisku nadmiernych oscylacji i tzw. *jitteru* (szarpania nastaw). Drastyczny spadek znaczenia kary za utyk (`p_factor_stall`) świadczy o tym, że w trakcie zoptymalizowanego treningu zjawisko zablokowania wału i palenia uzwojeń praktycznie przestało występować, dzięki czemu algorytm mógł zmniejszyć tę wagę na rzecz parametrów precyzyjnego pozycjonowania. Ostatecznie, ponad dwukrotne zwiększenie nagrody za prędkość (`r_velocity`) ustabilizowało zachowanie agenta bezpośrednio w wyznaczonym punkcie pracy.

6 Testy odporności na zakłócenia i weryfikacja końcowa (26.05.2026 – 02.06.2026)

Ostatnia faza projektu poświęcona była walidacji odporności optymalnie przeszkolonego agenta (wykorzystującego najlepsze parametry sieci oraz zoptymalizowaną funkcję celu) na celowo wprowadzany szum pomiarowy i procesowy. Testy przeprowadzono najpierw dla każdego źródła zakłóceń niezależnie (Tab. 1), wyznaczając maksymalne odchylenia standardowe szumu, przy których układ zachowywał stabilność i pełną zdolność regulacji. Eksperymenty pozwoliły na zdefiniowanie następujących granic tolerancji systemu:

- **Szum prądu:** pełna dokładność zachowana nawet przy ekstremalnych, sztucznych zakłóceniach sięgających 100 A.
- **Szum prędkości obrotowej:** tolerancja na zakłócenia do poziomu 1 rad/s.
- **Szum napięcia zasilania:** tolerancja do poziomu 0,1 V.
- **Szum momentu obciążenia zewnętrznego:** tolerancja do poziomu 0,01 Nm.

Tabela 1: Wpływ poziomu szumów różnych sygnałów na wskaźnik ITAE oraz przeregulowanie (Overshoot).

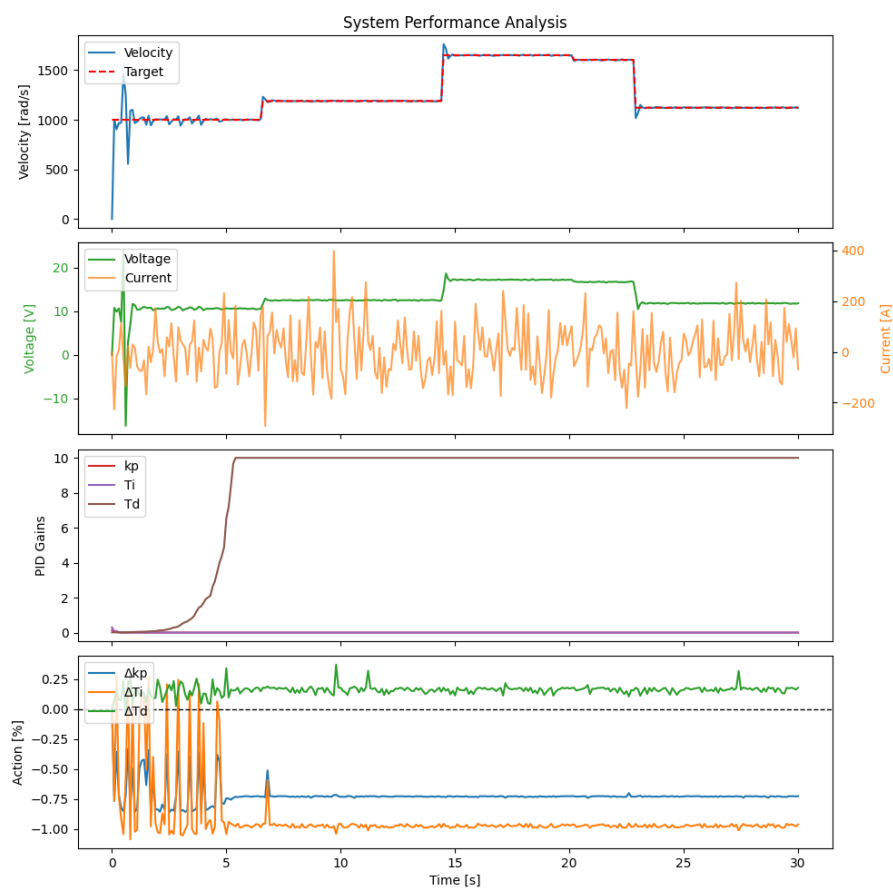
Noise level	ITAE	Overshoot [%]
Voltage [V]		
0.001	1802.427	44.143
0.01	1190.396	44.368
0.1	2557.032	46.958
1.0	5169.980	61.979
10.0	46412.695	30.726
Velocity [rad]		
0.1	1285.562	44.148
1.0	3358.430	43.779
10.0	6097.760	42.513
100.0	37056.769	43.639
Tl (load) [Nm]		
0.001	2430.610	42.785
0.01	1175.273	41.236
0.1	9167.958	44.535
1.0	82566.393	62.197
Current [A]		
0.1	2208.870	51.236
1.0	1377.891	61.849
10.0	1378.498	62.579
100.0	1835.064	66.664

Inicjalna interpretacja zachowania modelu potwierdziła, że regulacja zachowuje satysfakcjonującą jakość, o ile odchylenia standardowe wprowadzanych zakłóceń nie przekraczają wyżej wymienionych progów. W końcowym etapie przeprowadzono test integralny, poddając układ jednoczesnemu działaniu wszystkich zakłóceń granicznych (Rysunek 9). Układ poddał się rygorystycznej ewaluacji, a uzyskane wskaźniki jakości kontroli zestawiono w Tabeli 2.

Tabela 2: Wskaźniki jakości regulacji układu w warunkach pełnego zaszumienia.

Metryka jakości	Wartość
Wskaźnik ISE (<i>Integral of Squared Error</i>)	156569,050
Wskaźnik IAE (<i>Integral of Absolute Error</i>)	431,743
Wskaźnik ITAE (<i>Integral of Time-weighted Absolute Error</i>)	2042,760
Błąd w stanie ustalonym (E_{SS})	2,003
Maksymalne przeregulowanie (<i>Max Overshoot</i>)	45,908%
Średni czas regulacji (<i>Mean Settling Time</i>)	0,840 s

Pomimo skrajnie trudnych warunków i jednoczesnej obecności szumów we wszystkich kanałach informacyjnych, średni czas regulacji na poziomie 0,84 sekundy oraz powrót do niskiego błędu ustalonego potwierdzają wysoką jakość sterowania adaptacyjnego. Wykazano, że wyszkolony agent RL skutecznie



Rysunek 9: Odpowiedź skokowa zoptymalizowanego układu (Bayes + Aim) w warunkach jednoczesnego zaszumienia wszystkich kanałów pomiarowych i procesowych.

identyfikuje stan dynamiczny silnika BLDC i potrafi na bieżąco korygować nastawy struktury ISA-PID, realizując założone cele projektowe.